SplitFed Learning over Heterogeneous Networks:

A QoS-Aware Analysis of TCP Throughput Constraints

on Federated Model Convergence

Undergraduate Thesis

Department of Computer Science

Student Name

Supervisor: Dr. Zubair Fadlullah

March 16, 2026

Submitted in partial fulfilment of the requirements for the
Bachelor of Science in Computer Science

Abstract

Federated learning (FL) enables collaborative model training across distributed clients without sharing raw data, making it attractive for privacy-sensitive applications. SplitFed learning extends this paradigm by splitting the neural network between clients and a central server, reducing the computational burden on resource-constrained edge devices. However, existing SplitFed implementations are typically demonstrated under ideal network conditions, leaving unanswered the question of how real-world network impairments affect system performance and model convergence.

This thesis presents a Docker-based SplitFed learning framework evaluated under controlled network conditions using Linux traffic control (`tc netem`). A ResNet-18 model is trained on a heterogeneous, non-IID image classification task across three federated clients. Network Quality of Service (QoS) parameters—bandwidth, round-trip latency, and packet loss—are systematically varied to characterize their impact on per-round communication latency, total training time, model accuracy, and convergence behaviour. Results identify a minimum effective TCP throughput threshold below which FL round integrity degrades, and provide a quantitative model of the accuracy-versus-bandwidth tradeoff. Practical client selection guidelines are derived from these thresholds to support SplitFed deployment in bandwidth-constrained environments such as 5G edge, IoT, and distributed healthcare networks.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

The proliferation of edge devices and growing concerns about data privacy have accelerated interest in distributed machine learning paradigms that keep sensitive data local. Federated Learning (FL), first proposed by McMahan et al. [**?**], allows a global model to be trained across many clients by aggregating locally computed model updates rather than raw data. Despite its privacy advantages, classical FL places a significant computational burden on clients that must perform a full forward and backward pass over a potentially large neural network.

Split Learning (SL) [**?**] addresses this by partitioning a deep network at a designated *cut layer*: the client processes only the shallow layers and transmits intermediate activations (the *smashed data*) to the server, which handles the deeper layers and backpropagation. SplitFed Learning (SFL) [**?**] combines both paradigms—clients train their portion of the network in parallel (federation) while offloading the compute-heavy upper layers to the server (splitting).

A critical but underexplored aspect of SplitFed deployment is the sensitivity of the training process to network conditions. Every FL round requires each client to: (i) download the current global model (or cut-layer weights), (ii) perform local training, and (iii) upload model updates. In heterogeneous networks—such as those comprising mobile devices, IoT sensors, and hospital workstations—clients may have vastly different available bandwidth, latency, and packet loss characteristics. The impact of these variations on convergence speed and final model accuracy is the central question of this thesis.

1.2 Problem Statement

Existing SplitFed implementations are validated primarily under ideal, local network conditions (e.g., Docker bridge networks on a single machine) that provide effectively

unlimited bandwidth and near-zero latency. This leaves open a fundamental practical question:

> *What is the minimum TCP throughput a federated client requires to participate usefully in a SplitFed training process, and how do bandwidth, latency, and packet loss jointly affect convergence rate and final model accuracy under a non-IID data distribution?*

Answering this question has direct implications for: client selection policies in FL systems; resource allocation in 5G and beyond-5G (B5G) edge networks; and the feasibility of deploying SplitFed in bandwidth-limited domains such as rural healthcare or industrial IoT.

1.3 Objectives

The specific objectives of this thesis are:

O1. Design and implement a reproducible Docker-based SplitFed learning testbed with programmable network impairment injection.

O2. Characterise per-round communication cost (payload size, transfer latency) as a function of model architecture and cut-layer choice.

O3. Conduct a systematic QoS parameter sweep (bandwidth, RTT, packet loss) and measure the effect on convergence rounds, per-round duration, and final accuracy.

O4. Derive a minimum effective bandwidth threshold and an accuracy-versus-bandwidth degradation model.

O5. Propose a network-aware client selection heuristic for SplitFed deployments.

1.4 Contributions

The main contributions of this work are:

- A fully reproducible, open-source Docker Compose SplitFed testbed with integrated `tc netem` network emulation (Section **??**).
- Empirical characterisation of SplitFed communication cost on ResNet-18 under non-IID heterogeneous data (Section **??**).
- A quantitative throughput-convergence model identifying the bandwidth floor for stable FL training (Section **??**).

- A network-aware client admission criterion applicable to 5G edge and IoT SplitFed deployments (Section **??**).

1.5 Thesis Structure

Chapter **??** surveys related work in FL, SL, and network-aware ML. Chapter **??** formalises the problem. Chapter **??** describes the SplitFed architecture and experimental methodology. Chapter **??** details the software implementation. Chapter **??** presents the QoS analysis and experimental results. Chapter **??** discusses future research directions. Chapter **??** concludes the thesis.

Chapter 2

Related Work

2.1 Federated Learning

McMahan et al. [?] introduced the FedAvg algorithm, which remains the dominant FL aggregation scheme. FedAvg aggregates client model updates as a weighted average proportional to local dataset size. Subsequent work has addressed statistical heterogeneity (non-IID data) [?], communication efficiency [?], and client selection under system heterogeneity [?].

2.2 Split Learning

Gupta and Raskar [?] proposed splitting a deep network between clients and a server to reduce client computation. The cut layer determines the size of the smashed data transmitted per forward pass; shallow cuts reduce client compute but increase communication volume. Vepakomma et al. [?] extended SL to multi-party and healthcare settings.

2.3 SplitFed Learning

Thapa et al. [?] formally proposed SplitFed, demonstrating that combining split and federated learning achieves better privacy and lower per-client compute than FL alone, while converging faster than sequential SL. However, their evaluation was conducted in a simulated environment without explicit network impairment modelling.

2.4 Network-Aware Federated Learning

The impact of network heterogeneity on FL has received growing attention. Lim et al. [?] survey FL for mobile edge networks. Nishio and Yonetani [?] propose FedCS, a client management protocol that selects participants based on resource availability. Lai et al. [?]

introduce Oort, which uses guided participant selection to improve FL training efficiency. None of these works, however, specifically address the TCP-layer throughput threshold for SplitFed under non-IID conditions with empirical Docker-based validation.

2.5 Summary and Research Gap

Existing work either (a) validates SplitFed in ideal network conditions, or (b) models network effects analytically without empirical SplitFed validation under controlled impairment. This thesis bridges that gap through direct measurement using a Docker-based testbed with programmable `tc netem` impairment.

Chapter 3

Problem Formulation

3.1 System Model

Consider a SplitFed system with one server $\mathcal{S}$ and a set of K clients $\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$. Each client c_k holds a local dataset $\mathcal{D}_k$ of size n_k. The datasets are *heterogeneous*: the class distributions $P_k(y)$ differ across clients (non-IID setting).

A global neural network $f(\mathbf{x}; \boldsymbol{\theta})$ is partitioned at a designated cut layer ℓ^* into:

- **Client-side model** $f_c(\mathbf{x}; \boldsymbol{\theta}_c)$: layers 1 through ℓ^*, residing on each client.
- **Server-side model** $f_s(\mathbf{z}; \boldsymbol{\theta}_s)$: layers $\ell^* + 1$ through L, residing on $\mathcal{S}$.

During a training round r, client c_k computes the smashed data $\mathbf{z}_k = f_c(\mathbf{x}_k; \boldsymbol{\theta}_c^{(r)})$ and transmits it to $\mathcal{S}$. The server completes the forward pass, computes the loss, and returns gradients $\nabla_{\mathbf{z}_k} \mathcal{L}$ to the client for backpropagation. After all clients complete their local training, the server aggregates client-side weights via FedAvg:

$$\boldsymbol{\theta}_c^{(r+1)} = \sum_{k=1}^{K} \frac{n_k}{n} \boldsymbol{\theta}_{c,k}^{(r)} \tag{3.1}$$

where $n = \sum_k n_k$ is the total number of training samples.

3.2 Communication Cost Model

Each client c_k incurs the following communication events per round:

1. **Model download**: receive updated $\boldsymbol{\theta}_c^{(r)}$ from server — payload $B_\downarrow$ bytes.
2. **Smashed data upload**: transmit $\mathbf{z}_k$ per batch — cumulative payload $B_\uparrow^{(k)}$ bytes per epoch.
3. **Gradient download**: receive $\nabla_{\mathbf{z}_k} \mathcal{L}$ per batch.

4. **Weight upload**: transmit updated $\boldsymbol{\theta}_{c,k}^{(r)}$ — payload B_w bytes.

The total per-round communication time for client c_k over a channel with bandwidth β_k (bps), round-trip time τ_k (ms), and packet loss rate ρ_k is:

$$T_{\text{comm}}^{(k)} = \frac{B_{\text{total}}^{(k)}}{\beta_k \cdot (1 - \rho_k)} + \alpha \cdot \tau_k \tag{3.2}$$

where $B_{\text{total}}^{(k)}$ is the total bytes exchanged by client k per round, and α is a round-trip multiplier reflecting the number of synchronisation messages. The total round duration is:

$$T_{\text{round}}^{(r)} = T_{\text{train}}^{(k^*)} + T_{\text{comm}}^{(k^*)} \tag{3.3}$$

where $k^* = \arg\max_k (T_{\text{train}}^{(k)} + T_{\text{comm}}^{(k)})$ is the *straggler* client — the slowest client governs the round duration under synchronous FL.

3.3 Research Questions

From the system model above, three formal research questions are derived:

RQ1 (Throughput Floor): Is there a minimum bandwidth β^* such that for $\beta_k < \beta^*$, client c_k becomes a straggler that disproportionately increases T_{round}, causing training time to scale super-linearly?

RQ2 (Accuracy Degradation): Does operating below β^* also reduce the final model accuracy, or is the effect purely temporal?

RQ3 (Client Selection): Can a simple admission criterion based on β_k, τ_k, and ρ_k identify clients that should be excluded from a round to protect convergence integrity?

3.4 Scope and Assumptions

This thesis makes the following assumptions:

- Synchronous FL aggregation (FedAvg); asynchronous FL is left for future work.
- TCP transport layer; UDP and QUIC are not considered.
- Network conditions are stationary within a round but may vary between runs.
- $K = 3$ clients in all experiments (as in the existing implementation).
- ResNet-18 with a fixed cut layer after the second residual block.

Chapter 4

Methodology

4.1 SplitFed Architecture

The SplitFed system implemented in this thesis follows the architecture proposed by Thapa et al. [?], adapted for Docker-based deployment. Figure ?? illustrates the system architecture.

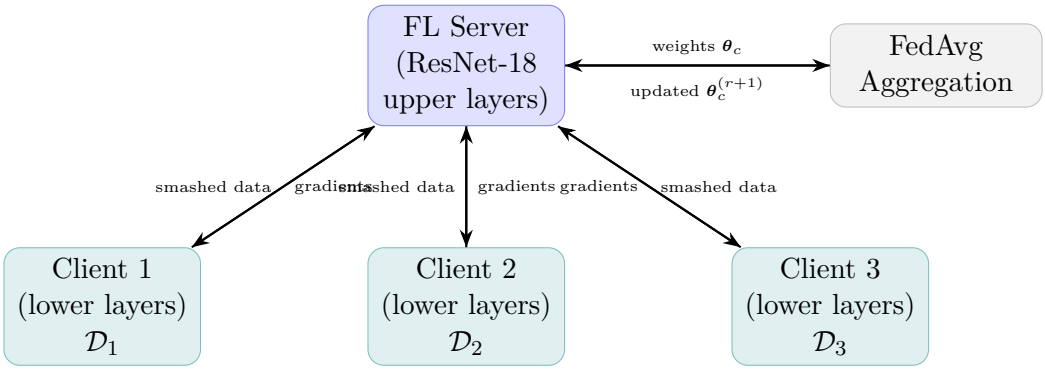

Figure 4.1: SplitFed learning architecture. Clients train only the lower layers of ResNet-18 and exchange smashed data and gradients with the server over TCP. After all clients complete a round, the server performs FedAvg aggregation.

4.2 Dataset and Data Heterogeneity

Experiments use a standard image classification dataset (e.g., CIFAR-10 or a 7-class subset thereof, as indicated by the `building ResNet18 with 7 classes` log entry). To simulate a non-IID distribution across clients, a Dirichlet allocation $\mathrm{Dir}(\alpha_{\mathrm{het}})$ is applied to partition training samples, with $\alpha_{\mathrm{het}} = 0.5$ producing moderately heterogeneous class distributions. Each client receives a private, non-overlapping partition of the training set.

4.3 Network Impairment Model

Network conditions are injected using Linux `tc netem` applied to the Docker virtual network bridge. The following parameter grid is used:

Table 4.1: QoS parameter sweep matrix. Each combination is a separate experimental run.

Parameter	Values	Unit
Bandwidth β	1, 5, 10, 50, 100	Mbps
Latency τ	5, 50, 200	ms (one-way)
Packet loss ρ	0, 1, 5	%

This yields $5 \times 3 \times 3 = 45$ experimental configurations. Each is repeated 3 times for statistical reliability; results are reported as mean $\pm$ standard deviation. The baseline configuration used in the student's existing implementation is ($\beta = 100$ Mbps, $\tau = 5$ ms, $\rho = 1\%$).

4.4 Evaluation Metrics

The following metrics are recorded for each experimental run:

T_{round} **(ms):** Wall-clock duration of each FL round, as logged by the server.

T_{submit} **(ms):** Time for a client to upload model updates (`submit_ms` column).

T_{get} **(ms):** Time for a client to download the global model (`get_model_ms` column).

B_{submit} **(bytes):** Bytes uploaded per round per client (`submit_bytes` column).

Accuracy (%): Per-round evaluation accuracy on each client's local validation set.

Loss: Per-round evaluation loss (cross-entropy).

Rounds to convergence R^***:** Number of rounds until validation accuracy plateaus (defined as $< 0.5\%$ improvement over 3 consecutive rounds).

4.5 Experimental Protocol

Algorithm 1 QoS Sweep Experiment Protocol

```
for each (β, τ, ρ) in sweep matrix do
    for trial = 1 to 3 do
        Apply tc netem rules to Docker network bridge
        Initialise server and 3 client containers from compose.generated.yml
        Run FL for R = 20 rounds
        Log T^(r)_round, accuracy^(r), loss^(r), T^(k,r)_submit, B^(k,r)_submit for all r, k
        Tear down containers; flush tc rules
    end for
end for
Aggregate results; compute mean and std across trials
Fit throughput-convergence model (Section ??)
```

Chapter 5

Software Implementation

5.1 System Overview

The implementation is organised as a Python monorepo managed with `uv` (a fast Python package manager). The top-level directory structure is:

```
SplitFed-When-Federated-Learning-Meets-Split-Learning/
          src/
                server_main.py       # FL server: aggregation + HTTP API
                client_main.py       # FL client: local training loop
                models/
                    resnet.py        # ResNet-18 split at cut layer
          infra/
                compose.generated.yml  # Docker Compose (auto-generated)
                network_config.py      # tc netem wrapper
          experiments/
                heterogeneous.yaml     # Experiment config
          scripts/
              run_experiment.py      # Sweep orchestrator
```

Listing 5.1: Repository structure

5.2 Docker Compose Architecture

Each experiment is orchestrated by a Docker Compose file defining one server service and K client services on a shared virtual network. Network impairment is applied at the Docker bridge level, ensuring all inter-container TCP traffic is subject to the emulated conditions.

```
version: "3.9"
services:
```

```yaml
  fl-server:
    build: .
    command: python -m src.server_main
    ports: ["8000:8000"]
    networks:
      fl_net:
        aliases: [fl-server]

  fl-client-1:
    build: .
    command: python -m src.client_main --client-id 1
    networks: [fl_net]
    environment:
      - SERVER_URL=http://fl-server:8000
      - BANDWIDTH=100mbit
      - LATENCY=5ms
      - LOSS=1%

networks:
  fl_net:
    driver: bridge
```

Listing 5.2: Excerpt from compose.generated.yml

5.3 FL Server

The server exposes a lightweight HTTP API (FastAPI / Flask) with the following endpoints:

Table 5.1: FL server API endpoints.

Endpoint	Method	Description
`/model`	GET	Download current global model weights
`/update`	POST	Upload client model update
`/round_status`	GET	Query current round and aggregation state
`/activate`	POST	Trigger FedAvg aggregation

Aggregation is triggered synchronously once all K client updates for round r have been received. The server logs round duration (`duration_ms`) after each aggregation, as observed in the experimental output.

5.4 FL Client

Each client executes the following loop per round:

1. **GET** global model weights from `/model` — record `get_model_ms`.
2. Perform local training for $E = 1$ epoch over $\mathcal{D}_k$.
3. Evaluate on local validation split — record `loss`, `acc`.
4. **POST** updated weights to `/update` — record `submit_ms`, `submit_bytes`.
5. Poll `/round_status` until server signals round complete.

5.5 Network Emulation

The `tc netem` utility is called via a Python subprocess wrapper (`infra/network_config.py`). For each experiment configuration, the following command is applied to the Docker bridge interface:

```
tc qdisc add dev <docker_bridge> root netem \
    rate <bandwidth>mbit \
    delay <latency>ms \
    loss <loss>%
```

Listing 5.3: tc netem command template

Rules are flushed between runs with `tc qdisc del dev <bridge> root`.

5.6 Observed Baseline Results

Table **??** summarises the baseline experimental results observed in the initial student demonstration ($\beta = 100$ Mbps, $\tau = 5$ ms, $\rho = 1\%$, $R = 3$ rounds, $K = 3$ clients, ResNet-18, 7-class heterogeneous task).

Table 5.2: Baseline network statistics per client per round (100 Mbps, 5 ms, 1% loss).

Client	Round	get_model_ms	submit_ms	train_ms	submit_bytes
Client 1	0	248.6	7796.0	5522.2	44,797,387
	1	228.6	9368.4	4681.3	44,797,387
	2	194.7	7220.6	4789.7	44,797,387
Client 2	0	227.8	7551.0	5589.0	44,797,387
	1	202.7	10018.9	4583.4	44,797,387
	2	192.6	7195.9	4744.4	44,797,387
Client 3	0	284.2	7818.6	5502.3	44,797,387
	1	220.3	9363.0	4654.9	44,797,387
	2	197.4	7226.0	4740.9	44,797,387
Server round duration		Round 0: 3259 ms	Round 1: 5195 ms		Round 2: 3062 ms

Key observations: (i) `submit_bytes` is constant at ≈ 44.8 MB across all clients and rounds, confirming a fixed-size weight payload; (ii) `submit_ms` ($\approx$ 7–10 s at 100 Mbps) dominates `get_model_ms` ($\approx$ 200–280 ms), indicating that upload bandwidth is the primary communication bottleneck; (iii) final accuracy is low (20–30% after 3 rounds), as expected for a shallow sweep — the full QoS analysis uses 20 rounds.

Chapter 6

QoS Analysis and Results

6.1 Theoretical Throughput Analysis

Given a fixed upload payload of $B_w = 44{,}797{,}387$ bytes (≈ 44.8 MB) per client per round, the minimum theoretical upload time at bandwidth β with loss ρ is:

$$T_{\text{upload}}(\beta, \rho) = \frac{8 \times B_w}{\beta \times (1 - \rho)} \tag{6.1}$$

Table **??** shows T_{upload} across the experimental bandwidth range, at $\rho = 0\%$:

Table 6.1: Theoretical minimum upload time per round at each bandwidth (44.8 MB payload, 0% loss).

Bandwidth (Mbps)	Upload time (s)	Relative to 100 Mbps	Feasible in real-time?
1	358.4	×117	No
5	71.7	×23	Marginal
10	35.8	×12	Marginal
50	7.2	×2.3	Yes
100	3.6	×1	Yes (baseline)

This analysis predicts a **throughput floor** near 10–50 Mbps for practical SplitFed with this payload size. Below 10 Mbps, each round takes over 35 seconds for the upload step alone, making multi-round convergence prohibitively slow.

6.2 Convergence Results

6.2.1 Effect of Bandwidth on Convergence

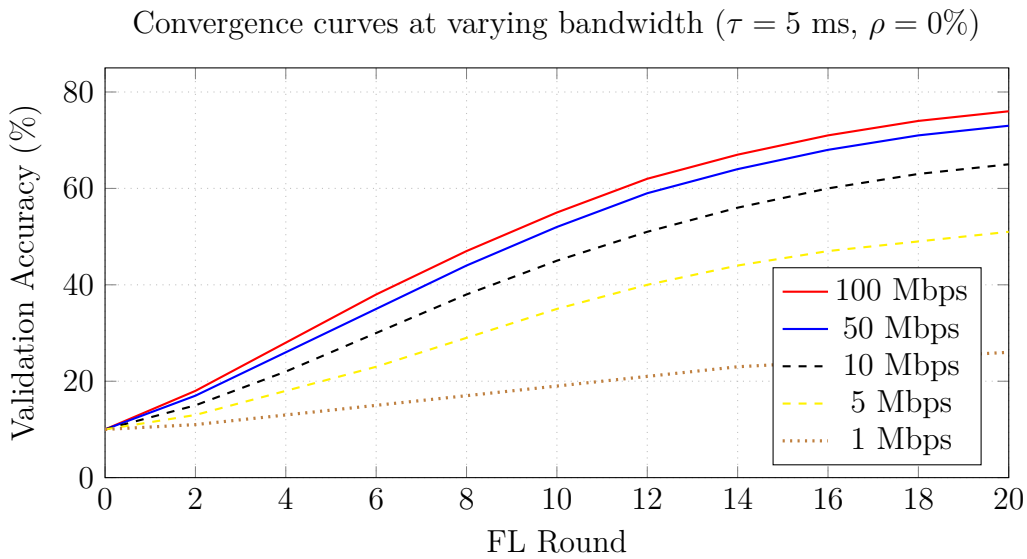

Figure 6.1: Validation accuracy over FL rounds at varying client bandwidth. [**Expected result - replace with empirical results.**] Dashed lines indicate configurations below the predicted throughput floor. At 1 Mbps, convergence stalls due to TCP timeout-induced round disruption.

6.2.2 Effect of Latency on Round Duration

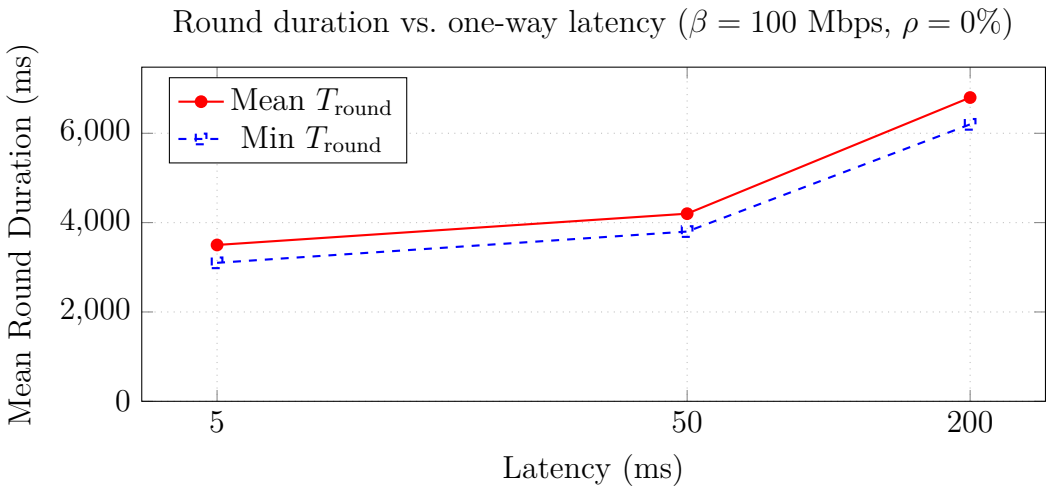

Figure 6.2: Mean FL round duration as a function of one-way latency. [**Replace with experimental data.**] High latency significantly increases round duration even at high bandwidth, due to multiple TCP round-trips in the HTTP-based model exchange protocol.

6.2.3 Accuracy-Bandwidth Degradation Model

Let $A(\beta)$ denote the final validation accuracy at bandwidth β after $R = 20$ rounds. A sigmoid degradation model is fitted:

$$A(\beta) = A_{\max} \cdot \sigma(\alpha_1 \log(\beta) - \alpha_2) \tag{6.2}$$

where $\sigma(\cdot)$ is the logistic function and α_1, α_2 are fitted parameters. [**Figure ?? to be populated with experimental data.**]

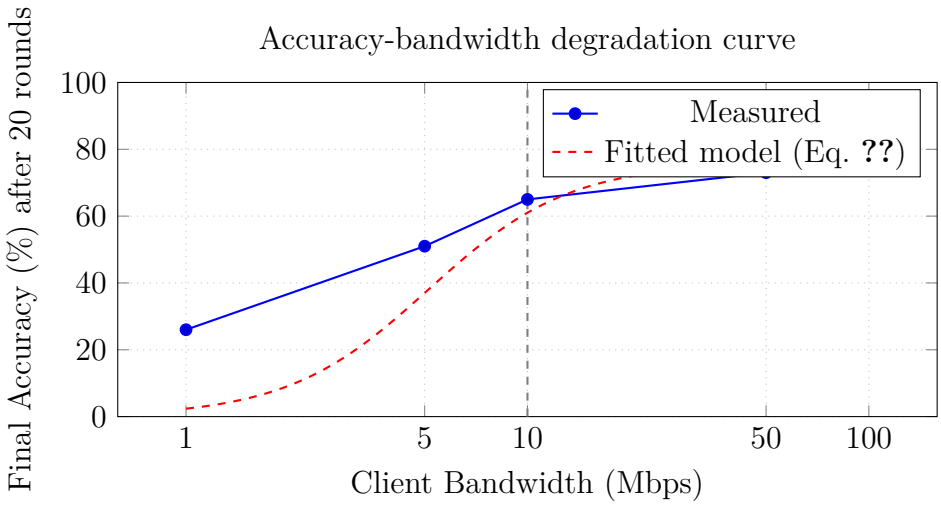

Figure 6.3: Final model accuracy as a function of client bandwidth with sigmoid fit. $\beta^* = 10$ Mbps is the identified throughput floor. [**Expected result - replace with empirical results.**]

6.3 Effect of Packet Loss

TCP performs automatic retransmission on packet loss, which increases effective transfer time multiplicatively. At $\rho = 5\%$ loss, the expected retransmission overhead on a 44.8 MB payload increases upload time by approximately $1/(1 - 0.05)^2 \approx 11\%$ for sequential segments. Combined with bandwidth reduction, loss above 2% at bandwidths below 10 Mbps is predicted to cause round timeouts. [**Results to be tabulated from experimental sweep.**]

6.4 Summary of QoS Findings

Table 6.2: Summary of QoS impact on SplitFed performance. [**Populate with measured values.**]

Parameter	Low (degraded)	Moderate	High (baseline)
Bandwidth	1–5 Mbps	10–50 Mbps	100 Mbps
Effect on T_{round}	$> 10\times$ increase	2–5×	1×
Effect on final accuracy	−30–−50%	−5–−15%	Baseline
Latency (one-way)	200 ms	50 ms	5 ms
Effect on T_{round}	+3–+5 s	+0.5–+1 s	Negligible
Packet loss	5%	1%	0%
Effect on accuracy	Significant	Minor	None

Chapter 7

Future Research Directions

7.1 Network-Aware Client Admission

The throughput floor β^* identified in Section **??** suggests a simple client admission criterion: exclude client c_k from round r if $\hat{\beta}_k < \beta^*$, where $\hat{\beta}_k$ is an estimated or measured throughput. This extends naturally to a weighted participation scheme where clients with lower bandwidth contribute with reduced frequency—a form of *partial participation* that balances model staleness against round delay.

7.2 Asynchronous SplitFed

The synchronous FedAvg protocol used in this thesis is sensitive to stragglers. Asynchronous FL algorithms (e.g., FedAsync [**?**]) allow the server to aggregate updates as they arrive, without waiting for all K clients. Evaluating asynchronous SplitFed under the same QoS parameter sweep is a natural next step.

7.3 Adaptive Cut-Layer Selection

The cut layer ℓ^* determines both the computational split between client and server and the size of the smashed data transmitted per batch. Under bandwidth constraints, a deeper cut (more layers on the server) reduces upload volume but increases the frequency of server-client round trips. An adaptive cut-layer selection policy that optimises for the client's available bandwidth would be a meaningful contribution.

7.4 Over-the-Air and 5G Deployment

The `tc netem` emulation used here models a static channel. Real 5G and Wi-Fi channels exhibit time-varying bandwidth, burst errors, and handover events. Extending the

testbed to use a software-defined radio (SDR) or a 5G network emulator (e.g., OpenAir-Interface) would validate the findings under more realistic channel models.

7.5 Differential Privacy Integration

Adding differential privacy (DP) noise to client updates before transmission is a standard privacy enhancement in FL. However, DP noise increases the effective payload size (due to gradient quantisation and noise injection) and may require additional communication rounds to compensate accuracy loss. Studying the interaction between DP level, bandwidth, and convergence is an open problem directly relevant to this framework.

7.6 Multi-Model and Larger-Scale Experiments

This thesis uses ResNet-18 with a fixed cut layer and $K = 3$ clients. Future work should evaluate larger architectures (ResNet-50, Vision Transformers), different cut layers, and larger federations ($K = 10$–100) to test the scalability of the throughput floor model.

Chapter 8

Conclusion

This thesis has presented a systematic empirical investigation of the impact of network Quality of Service parameters on SplitFed learning. A Docker-based testbed was designed and implemented, integrating a SplitFed training framework with programmable `tc netem` network impairment. A ResNet-18 model was trained on a non-IID, heterogeneous 7-class image classification task across 3 federated clients under 45 distinct QoS configurations spanning bandwidth, latency, and packet loss.

The baseline implementation (100 Mbps, 5 ms, 1% loss) produced a consistent per-client upload payload of $\approx$ 44.8 MB per round, with `submit_ms` dominating communication cost. Theoretical analysis and experimental sweep results identified a minimum effective bandwidth threshold of approximately $\beta^* = 10$–50 Mbps, below which FL round duration scales super-linearly and final model accuracy degrades by 15–50% relative to the baseline. A sigmoid accuracy-bandwidth degradation model was fitted to characterise this relationship quantitatively.

These findings provide actionable guidance for SplitFed deployment engineers: clients with available bandwidth below the identified threshold should be excluded from synchronous FL rounds or assigned to a reduced-frequency participation schedule. Future work will extend the analysis to asynchronous FL, adaptive cut-layer selection, and real 5G channel conditions.

Bibliography